Přírodovědecká fakulta

Univerzita Karlova

Algoritmy počítačové kartografie

Point Location Problem

Kristýna Antošová, Vít Kadlec

1.N-GKDPZ

16. 3. 2026, Praha

1 Zadání

Úloha č. 1: Geometrické vyhledávání bodu

Vstup: Souvislá polygonová mapa n polygonů $\{P_1, ..., P_n\}$, *analyzovaný bod* q.

Výstup: P_i, $q \in P_i$.

Nad polygonovou mapou implementujete Ray Crossing Algorithm pro geometrické vyhledání incidujícího polygonu obsahujícího zadaný bod q.

Nalezený polygon graficky zvýrazněte vhodným způsobem (např. vyplněním, šrafováním, blikáním). Grafické rozhraní vytvořte s využitím frameworku QT.

Pro generování nekonvexních polygonů můžete navrhnout vlastní algoritmus či použít existující geografická data (např. mapa evropských států).

Polygony budou načítány z textového souboru ve Vámi zvoleném formátu. Pro datovou reprezentaci jednotlivých polygonů použijte špagetový model.

V rámci zadání byly řešeny tyto bonusové úlohy:

Krok	**Hodnocení**
Detekce polohy bodu rozlišující stavy uvnitř, vně polygonu.	10b
Analýza polohy bodu (uvnitř/vně) metodou Winding Number Algorithm.	*+10b*
Ošetření singulárního případu u Winding Number Algorithm: bod leží na hraně polygonu.	*+5b*
Ošetření singulárního případu u Ray Crossing Algorithm: bod leží na hraně polygonu.	*+5b*
Ošetření singulárního případu u obou algoritmů: bod je totožný s vrcholem jednoho či více polygonů.	*+2b*
Zvýraznění všech polygonů pro oba výše uvedené singulární případy.	*+3b*
Rychlé vyhledání potenciálních polygonů (bod uvnitř min-max boxu).	*+10b*
Řešení pro polygony s dírami.	*+10b*
*Načtení vstupních dat ze *.shp.*	*+10b*
Max celkem:	**65b**

2 Popis a rozbor problému

2.1 Point Location Problem

Problém určení polohy bodu, známý také jako Point Location Problem, se zabývá zjišťováním prostorového vztahu mezi daným bodem a polygonem či množinou polygonů. Tento problém nachází uplatnění v celé řadě oblastí, nejenom v oblasti geoinformatiky, ale také v počítačové grafice, robotice či databázích.

K řešení této úlohy lze postupovat lokálně nebo globálně. Při lokální proceduře opakovaně zjišťujeme, zda dotazovaný bod leží uvnitř, vně nebo na hraně daného mnohoúhelníku. Při globální proceduře hodnotíme polohu bodu vůči celé množině mnohoúhelníků. Výsledkem analýzy je zjištění, že bod leží uvnitř jednoho z nich, vně všech, nebo se nachází na hraně či uzlu dvou a více polygonů.

Algoritmy řešící tento problém lze rozdělit podle tvaru polygonu. Pro konvexní mnohoúhelníky, tedy ty, které nemají vnitřní úhly větší než 180°, lze využít metody jako Half-plane Test nebo Ray Crossing. V geoinformatice a kartografii se však mnohem častěji setkáme s nekonvexními mnohoúhelníky, které mají alespoň jeden vnitřní úhel větší než 180°. Pro ty je vhodné použít algoritmy Winding Number nebo Ray Crossing.

2.2 Winding Number Method

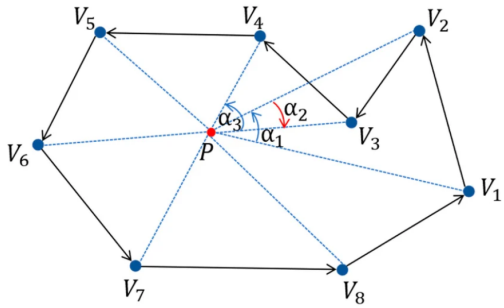
Metoda Winding Number, také nazývána jako metoda ovíjení, zjišťuje, kolikrát se polygon otočí kolem daného bodu q. Výsledná hodnota Ω je součet všech úhlů, které svírá pozorovaný bod q s jednotlivými hranami polygonu. Pokud je hodnota Ω nenulová, bod q leží uvnitř polygonu P. Pokud je rovna nule, bod leží mimo polygon:

$$\Omega(q, P) = \frac{1}{2\pi} \sum_{i=1}^{n} \omega(p_i, q, p_{i+1}) = \begin{cases} 1, & q \in P \\ 0, & q \notin P \end{cases} \tag{1}$$

Pro výpočet polohy bodu q vůči přímce dané hranou $p_i p_{i+1}$ pracujeme se směrovými vektory vedenými z bodu ke zkoumaným vrcholům. Přímka p rozděluje rovinu na levou polorovinu σ_l a pravou polorovinu σ_r. To, zda je výsledek přičítán či odečítán, závisí na polorovině, což se zjišťuje výpočtem determinantu, který funguje jako ekvivalent vektorového součinu. V levé polorovině se úhly přičítají (rotace proti směru hodinových ručiček), v pravé se odečítají (rotace po směru). Výsledná hodnota Ω je udávána v násobcích 2π.

$$t = \begin{vmatrix} p_x & p_y \\ v_x & v_y \end{vmatrix} = \begin{cases} > 0, & q \in \sigma_l \\ = 0, & q \in p \\ < 0, & q \in \sigma_r \end{cases} \tag{2}$$

Nevýhodou tohoto algoritmu oproti metodě Ray Crossing je celkově vyšší časová náročnost, přináší nám však lepší ošetření singulárních případů.

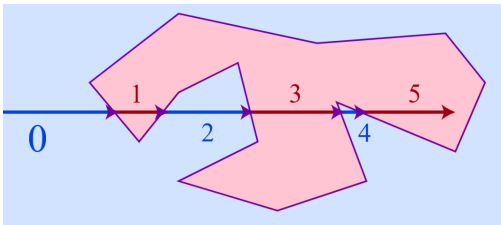


Obrázek 1: *Winding Number Algorithm (zdroj: Liu et al. 2023)*

2.3 Ray Crossing Algorithm

Při této metodě je zkoumaným bodem q vedena v libovolném směru polopřímka r. Určující je počet průsečíků této polopřímky s hranami polygonu P. Počet průsečíků označujeme k a pokud je jeho hodnota lichá, bod leží uvnitř polygonu P. Pokud je sudá, bod leží vně polygonu:

$$k \bmod 2 = \begin{cases} 0, & q \notin P \\ 1, & q \in P \end{cases} \tag{3}$$



Obrázek 2: *Ray Crossing Algorithm (zdroj: Kumar 2024)*

3 Singulární případy a jejich ošetření

3.1 Bod na vrcholu polygonu

U obou algoritmů je singulární případ, kdy je bod q totožný s některým vrcholem polygonu, ošetřen porovnáním souřadnic bodu a jednotlivých vrcholů pomocí metody `isPointOnVertex()`. Před samotným výpočtem Ω či k iterujeme přes všechny vrcholy polygonu a pokud jsou obě složky souřadnicového rozdílu menší než pixelová tolerance $TOL = 3\,\text{px}$, je vrácen výsledek -1:

$$|x_q - x_i| < \varepsilon \quad \wedge \quad |y_q - y_i| < \varepsilon \implies q \equiv p_i \tag{4}$$

3.2 Bod na hraně polygonu – Winding Number

U algoritmu Winding Number je detekce hrany provedena v metodách `getHalfPlane()` a `getAngle()` a je založena na dvou podmínkách, které musí platit současně. Za prvé musí bod q ležet na přímce $p_i p_{i+1}$, což se ověřuje normalizovaným determinantem t – determinant je vydělen délkou hrany, čímž se získá kolmá vzdálenost bodu od přímky v pixelech:

$$t = (x_{i+1} - x_i)(y_q - y_i) - (y_{i+1} - y_i)(x_q - x_i) \tag{5}$$

Za druhé musí být úhel ω_i mezi vektory z q do p_i a z q do p_{i+1} roven π, což zaručuje, že bod leží přímo mezi vrcholy hrany a nikoliv na jejím prodloužení. Obě podmínky jsou testovány s tolerancí $e = 0{,}05$:

$$\frac{|t|}{|p_{i+1} - p_i|} < e \quad \wedge \quad |\omega_i - \pi| < e \implies q \in \partial P \tag{6}$$

3.3 Bod na hraně polygonu – Ray Crossing

U algoritmu Ray Crossing je detekce hrany realizována pomocí dvou paprsků s opačnou orientací – levostranného r_l a pravostranného r_r. Souřadnice vrcholů polygonu jsou redukovány posunutím počátku souřadnicového systému do bodu q. Pro každou hranu zvlášť se spočítá počet průsečíků s polygonem, tedy k_l a k_r. Pokud bod leží na hraně polygonu, počet průsečíků se bude mezi dvěma paprsky lišit:

$$q = \begin{cases} \in \partial P, & k_l \,\%\, 2 \neq k_r \,\%\, 2 \\ \in P, & k_r \,\%\, 2 = 1 \\ \notin P, & \text{jinak} \end{cases} \tag{7}$$

3.4 Zvýraznění polygonů při singulárních případech

A aplikaci je zvýrazněn polygon obsahující bod a zároveň pokud bod leží na vrcholu nebo hraně sdílené více polygony, jsou zvýrazněny všechny takové polygony. Při detekci singularity se polygon přidá do seznamu zvýrazněných polygonů, ale iterace přes zbývající polygony pokračuje, aby byly nalezeny všechny polygony sdílející daný vrchol či hranu. Jejich indexy jsou průběžně ukládány do seznamu `highlighted` v `MainForm` a předány metodě `setHighlighted()` třídy `Draw`. Zvýrazněné polygony mají tyrkysovou výplň (`Qt.GlobalColor.cyan`).

4 Popis algoritmů formálním jazykem

4.1 Winding Number Algorithm

Tato sekce uvádí pseudokódy, které věrně odrážejí implementační detaily algoritmů.

Algorithm 1 Winding Number

Input: point q, polygon *pol*
Output: -1 (vertex) | -2 (edge) | 0 (outside) | 1 (inside)

```
for each vertex p_i of polygon do
    if |x_q − x_i| < ε  and  |y_q − y_i| < ε then return −1
    end if
end for
ω_sum ← 0
for each edge (p_i, p_{i+1}) do
    t ← getHalfPlane(q, p_i, p_{i+1})
    ω_i ← getAngle(q, p_i, p_{i+1})
    ℓ ← |p_{i+1} − p_i|
    if ℓ > 0  and  |t|/ℓ < e  and  |ω_i − π| < e then return −2
    end if
    if t > 0 then ω_sum += ω_i
    else ω_sum −= ω_i
    end if
end for
if round(ω_sum/2π) ≠ 0 then return 1
else return 0
end if
```

4.2 Ray Crossing Algorithm

Algorithm 2 Ray Crossing

Input: point q, polygon pol
Output: -1 (vertex) | -2 (edge) | 0 (outside) | 1 (inside)

```
for each vertex p_i of polygon do
    if |x_q − x_i| < ε  and  |y_q − y_i| < ε then return −1
    end if
end for
k_l ← 0, k_r ← 0
for each edge (p_i, p_{i+1}) in coordinates shifted by q do
    if (y_{i+1} − y_i) = 0 then continue
    end if
    x_m ← (x_{i+1} · y_i − x_i · y_{i+1})/(y_{i+1} − y_i)
    if (y_{i+1} < 0) ≠ (y_i < 0)  and  x_m < 0 then k_l += 1
    end if
    if (y_{i+1} > 0) ≠ (y_i > 0)  and  x_m > 0 then k_r += 1
    end if
end for
if k_l mod 2 ≠ k_r mod 2 then return −2
end if
if k_r mod 2 = 1 then return 1
else return 0
end if
```

5 Rychlé vyhledávání pomocí min-max boxu

Před spuštěním samotných algoritmů RC a WN je pro každý polygon testováno, zda bod q leží uvnitř jeho osově zarovnaného ohraničujícího obdélníku, tedy min-max boxu. Tento obdélník je definován minimálními a maximálními souřadnicemi x a y ze všech vrcholů polygonu. Pokud bod neleží v žádném min-max boxu, algoritmy RC a WN se nespouštějí a uživatel je na tuto situaci upozorněn. Tím je výrazně snížena výpočetní náročnost.

6 Řešení pro polygony s dírami

Další řešenou úlohou bylo správné ošetření polygonů, které obsahují díry. Řešení této problematiky je rozděleno mezi třídy `Draw` a `MainForm`.

6.1 Vizualizace děr (třída Draw)

Při načítání ze souboru shapefile jsou pro každý dotčený polygon zpracovány nejen jeho vnější hranice (exterior rings), ale i všechny případné interior rings, tedy díry. Z každého polygonu je vytvořen objekt `QPainterPath` s pravidlem vykreslování `OddEvenFill`. Návrh tohoto konkrétního postupu pro korektní vizualizaci děr pomocí knihovny Qt byl konzultován s umělou inteligencí, konkrétně LLM Claude Sonnet 4.6. Exterior ring je přidán jako první subpath a po něm následují všechny interior ringy. Knihovna Qt díky pravidlu Even-Odd místa vnitřního překryvu nevyplní.

Exterior polygony jsou ukládány do seznamu `__shp_polygons` a interior ringy jsou ukádány odděleně do proměnné `__shp_holes` jako seznam seznamů, tedy jeden vnořený seznam pro každý načtený polygon. Vykreslovací cesty se ukládají v proměnné `__shp_paths`. V samotné metodě `paintEvent` se následně volá `drawPath()` místo `drawPolygon()`, což vede ke korektnímu vykreslení děr.

6.2 Analýza polohy bodu u polygonů s dírami (třída MainForm)

Metoda `analyzePointAndPositionClick()` využívá getter `getPolygonsData()`, který vrací seznam tuplů ve formátu `(exterior QPolygonF, [hole QPolygonF, ...])`. Návrh vnořené logiky pro vyhodnocování polohy bodu vůči dírám a ošetření hraničních singularit byl v rámci tohoto projektu vypracován za asistence LLM Claude Sonnet 4.6. Analýza přítomných děr probíhá následovně:

- Pokud výpočet nad dírou vrátí -1 nebo -2 – bod leží na hranici díry, polygon je zvýrazněn a status bodu je nastaven přímo uvnitř iterace přes díry.
- Pokud díra vrátí 1 – bod leží uvnitř díry, příslušný polygon se přeskočí a bod je hodnocen jako bod mimo polygon.
- Pokud žádná z děr nevrátí pozitivní shodu – bod leží skutečně uvnitř polygonu, výsledek je 1.

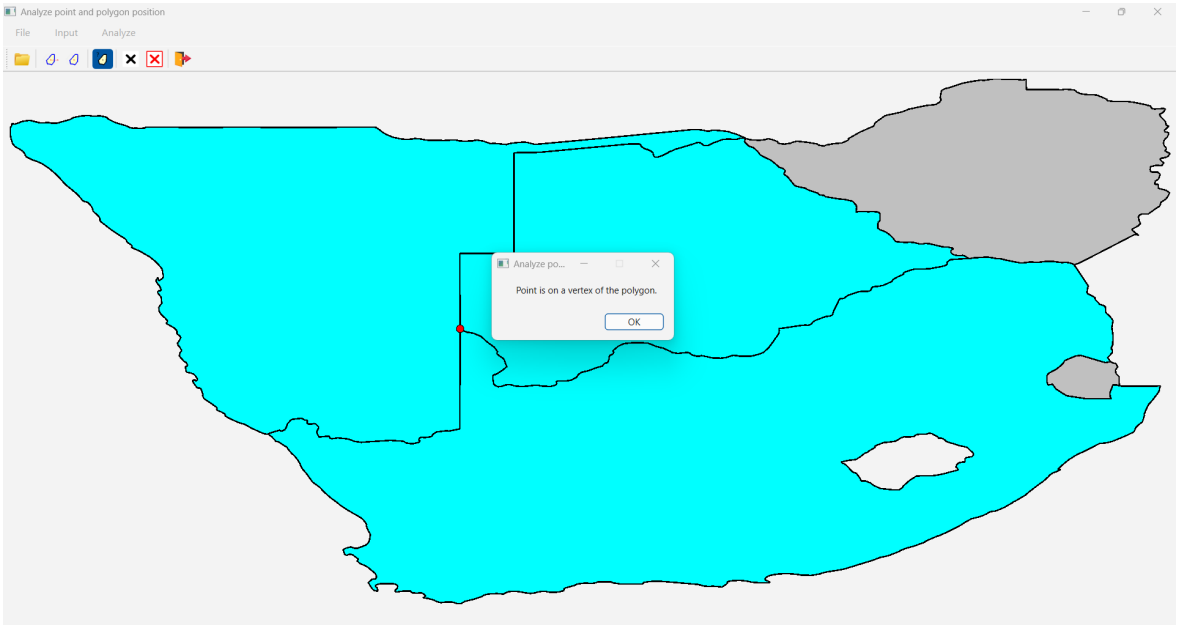
7 Vstupní data a Aplikace

Vstupními daty je polygonová vrstva ve formátu shapefile, která je otevřena pomocí knihovny GeoPandas. Geografické souřadnice jsou převedeny do souřadnic obrazovky – přizpůsobeny velikosti okna aplikace. Osa y je přitom otočena, protože v geografii roste směrem nahoru, zatímco na obrazovce směrem dolů. Po načtení shapefilu se aplikace automaticky přepne do režimu vkládání bodu a tlačítko *Point/Polygon* je deaktivováno, aby nebylo možné manuálně kreslit polygon přes data ze shapefilu. Aplikace byla testována na datech jižní Afriky ve formátu shapefile, obsahujících polygony administrativních hranic. Na těchto datech bylo ověřeno správné fungování obou algoritmů včetně detekce singulárních případů a zpracování polygonů s dírami.

Po spuštění skriptu *MainForm.py* se otevře okno aplikace. Aplikace má tři záložky. V záložce *File* jsou umístěny funkce *Open* a *Exit*, v záložce *Input* funkce *Point/Polygon*, *Clear Results* a *Clear Data* a poslední záložka *Analyze* obsahuje funkce algoritmů *Winding Number* a *Ray Crossing*. Ikony všech funkcí jsou umístěny na nástrojové liště.

Po kliknutí na funkci *Open* se otevře nabídka pro vybrání souboru ve formátu shapefile. Pro umístění bodu na mapu stačí kliknout na libovolné místo v aplikaci. Jestliže není otevřen shapefile soubor a pokud je zakliknuta funkce *Point/Polygon*, lze kreslit vlastní polygon. Po umístění analyzovaného bodu a spuštění příslušné metody se otevře dialogové okno s informací o poloze bodu a zvýrazní se polygon, jemuž bod náleží či více polygonů, pokud je bod umístěn na jejich hranu či vrchol.

Kliknutím na *Clear Results* je smazán nakreslený bod a zvýraznění po kliknutí na *Clear Data* se smažou veškerá data a tlačítko *Point/Polygon* je znovu aktivováno. Aplikaci lze ukončit funkcí *Exit*.



Obrázek 3: *Ukázka Aplikace*

8 Dokumentace

8.1 Třída Algorithms (`algorithms.py`)

Třída `Algorithms` obsahuje logiku geometrické analýzy. Metody přijímají argumenty typu `QPointF` a `QPolygonF`.

- `isPointOnVertex(q, p)` – porovná souřadnice bodu q a vrcholu p s pixelovou tolerancí `TOL = 3.0`. Vrátí `True` pokud je bod v této toleranci.
- `getHalfPlane(q, p1, p2)` – vypočítá determinant pro test poloroviny. Kladná hodnota znamená levou polorovinu, záporná pravou. Nula znamená, že bod leží na přímce hrany.
- `getAngle(q, p1, p2)` – vypočítá absolutní úhel mezi vektory $q \to p_1$ a $q \to p_2$ pomocí `atan2`, normalizovaný do $(-\pi, \pi]$.
- `isPointInMinMaxBox(q, pol)` – testuje, zda bod q leží uvnitř ohraničujícího obdélníku polygonu. Slouží jako rychlý filtr před spuštěním složitějšího algoritmu.
- `getPointPolygonPositionWN(q, pol)` – analýza metodou Winding Number. Vrací -1 (vrchol), -2 (hrana), 0 (vně) nebo 1 (uvnitř).
- `getPointPolygonPositionRC(q, pol)` – analýza metodou Ray Crossing. Vrací -1 (vrchol), -2 (hrana), 0 (vně) nebo 1 (uvnitř).

8.2 Třída Draw (`draw.py`)

Třída `Draw` zajišťuje veškeré grafické operace.

- `loadShapefile(file_name)` – načítá SHP soubor přes GeoPandas podle zadané cesty, zpracovává jeho geometrie, normalizuje souřadnice a sestavuje `QPainterPath`. Po načtení SHP automaticky přepne do režimu vkládání bodu.
- `paintEvent(e)` – vykreslí polygony ze shapefilu pomocí metody `drawPath()`. Polygony, které byly algoritmem vyhodnoceny jako pozitivní, jsou zvýrazněny tyrkysovou výplní. Samotný analyzovaný bod q je vykreslen červenou barvou.
- `mousePressEvent(e)` – při kliknutí přidá vrchol manuálního polygonu nebo přesune bod q a resetuje zvýraznění.
- `setHighlighted(indices)` – nastaví seznam indexů zvýrazněných polygonů a překreslí canvas.
- `clearResults()` – smaže bod q a zvýraznění, polygony zachová.
- `clearData()` – smaže veškerá data včetně polygonů.
- `getPolygonsData()` – vrátí seznam tuplů `(exterior QPolygonF, [hole QPolygonF, ...])` pro analýzu, nebo manuální polygon s prázdným seznamem děr.

8.3 Třída MainForm (`MainForm.py`)

Třída `MainForm` řídí inicializaci hlavního okna a propojuje třídy `Draw` a `Algorithms`.

- `analyzePointAndPositionClick(method)` – sdílená analytická logika pro RC i WN. Iteruje přes polygony, aplikuje min-max filtr, spouští zvolený algoritmus, ošetřuje díry a předává výsledky do `Draw`.
- `rayCrossingClick()` – zavolá `analyzePointAndPositionClick("rc")`.
- `windingNumberClick()` – zavolá `analyzePointAndPositionClick("wn")`.
- `openClick()` – otevře dialog pro výběr souboru, zavolá `loadShapefile(file_name)` třídy `Draw` a deaktivuje přepínač *Point/Polygon*.
- `changeStatusClick()` – přepne vstupní režim (bod / vrchol polygonu).
- `clearClick()` – zavolá `clearData()` a znovu aktivuje přepínač *Point/Polygon*.

9 Závěr

Pro řešenou úlohu bylo naimplementovo řešení využívající metody Ray Crossing a Winding Number pro řešení problému Point Location. Oba algoritmy úspěšně rozlišují prostorové stavy uvnitř, vně a na hranici polygonu. Singulární případy byly ošetřeny – detekce vrcholu je provedena geometricky na bázi porovnání souřadnic s pixelovou tolerancí, detekce hrany u algoritmů WN i RC byla řešena matematicky. Pro WN podmínkou normalizovaného t a $\omega \approx \pi$ s tolerancí $e = 0{,}05$, pro RC podmínkou různé parity k_l a k_r.

Jako bonusové úlohy bylo implementováno rychlé vyhledávání pomocí min-max boxu, načítání dat ze shapefile, zvýrazňování polygonů a řešení polygonů s dírami. Problematika děr je řešena vizuálně pomocí `QPainterPath` s pravidlem `OddEvenFill` a analyticky opakovaným voláním algoritmů na interior ringy v `MainForm`. Tímto postupem nebylo nutné zasahovat do třídy `Algorithms`.

Zdroje

[1] BAYER, T. (2026): *Point Location Problem.* Přednáška pro předmět Algoritmy počítačové kartografie, Katedra aplikované geoinformatiky a kartografie, Přírodovědecká fakulta UK. Dostupné z: `https://web.natur.cuni.cz/~bayertom`

[2] DE BERG, M., VAN KREVELD, M., OVERMARS, M., SCHWARZKOPF, O. (2000): *Computational Geometry.* Springer.

[3] KUMAR, A. (2024): *CG Notes – Ray Tracing: Ray-Polygon Intersection.* Dostupné z: `https://anirudh-s-kumar.github.io/CG-Notes/ray_tracing/`

[4] LIU, L., SUN, Y., JI, M., WANG, H., LIU, J. (2023): Efficient Construction of Voxel Models for Ore Bodies Using an Improved Winding Number Algorithm and CUDA Parallel Computing. *ISPRS International Journal of Geo-Information*, 12, 12, 473. `https://doi.org/10.3390/ijgi12120473`

[5] Qt Group (2025): *Qt Documentation.* `https://doc.qt.io/`